📌 Generic Functions in C++

> **A generic function** (also called a *template function*) defines a general set of operations that can be applied to **various data types**.
> The data type is supplied as a **template parameter**, allowing a single function definition to work with many concrete types.

- **Definition syntax**

Component	Description
template <class T>	Introduces a **type placeholder** T. class can be replaced by typename.
return-type func-name(parameter list)	Normal function header; the placeholder T may appear in the return type or parameters.
{ … }	Function body that must be **type-independent** (same logic for all T).

Example – swapping two values

```
#include <iostream>
using namespace std;

template <class X>
void swapargs(X &a, X &b) {
    X temp = a;
    a = b;
    b = temp;
}
int main() {
    int i = 10, j = 20;
    double x = 10.1, y = 23.3;
    char a = 'x', b = 'z';

    cout << "Original i, j: " << i << ' ' << j << '\n';
    cout << "Original x, y: " << x << ' ' << y << '\n';
    cout << "Original a, b: " << a << ' ' << b << '\n';

    swapargs(i, j);   // int specialization
    swapargs(x, y);   // double specialization
    swapargs(a, b);   // char specialization

    cout << "Swapped i, j: " << i << ' ' << j << '\n';
```

```
    cout << "Swapped x, y: " << x << ' ' << y << '\n';
    cout << "Swapped a, b: " << a << ' ' << b << '\n';
    return 0;
}
```

Output

```
Original i, j: 10 20
Original x, y: 10.1 23.3
Original a, b: x z
Swapped i, j: 20 10
Swapped x, y: 23.3 10.1
Swapped a, b: z x
```

- The compiler **instantiates** a concrete version for each used type (int, double, char).
- The generated version is called a **specialization** or **generated function**.

⚙️ Rules for Template Definition

- The template clause **must directly precede** the function definition.
- No other statements may appear between them.
- Example of an **invalid** placement:

```
template <class X>
int i;   // error — not a function definition
```

🔢 Function Templates with Multiple Type Parameters

> A template can accept **more than one** generic type by listing them comma-separated.

Syntax

```
template <class T1, class T2>
return-type func(T1 a, T2 b);
```

Example

```
#include <iostream>
using namespace std;

template <class type1, class type2>
void myfunc(type1 x, type2 y) {
    cout << x << ' ' << y << '\n';
}
int main() {
    myfunc(10, "I like C++");   // type1=int, type2=char*
    myfunc(98.6, 19L);           // type1=double, type2=long
    return 0;
}
```

Output

```
10 I like C++
98.6 19
```

🛠️ Explicit Specialization of Function Templates

> **Explicit specialization** provides a **hand-crafted version** of a template for a particular type, overriding the generic implementation.

New-style specialization syntax

```
template<> void swapargs<int>(int &a, int &b) {
    int temp = a; a = b; b = temp;
    cout << "Inside swapargs int specialization.\n";
}
```

- template<> signals a specialization.
- swapargs<int> names the type being specialized.

Full example

```
#include <iostream>
using namespace std;

// Generic version
template <class T>
```

```
void show(T a) {
    cout << "Generic version: " << a << endl;
}

// Explicit overload for int (regular function, not a template)
void show(int a) {
    cout << "Specialized version for int: " << a << endl;
}
int main() {
    show(10);      // calls int overload
    show(3.14);    // calls generic template
    show('A');     // calls generic template
    return 0;
}
```

Output

```
Specialized version for int: 10
Generic version: 3.14
Generic version: A
```

🔁 Overloading Template Functions

> Overloading can be performed **at the template level** by providing multiple templates with **different parameter lists**.

Example

```
#include <iostream>
using namespace std;

// One-parameter version
template <class X>
void f(X a) {
    cout << "Inside f(X a)\n";
}

// Two-parameter version
template <class X, class Y>
void f(X a, Y b) {
    cout << "Inside f(X a, Y b)\n";
}
```

```
int main() {
    f(10);          // matches first template
    f(10, 20);      // matches second template
    return 0;
}
```

Output

```
Inside f(X a)
Inside f(X a, Y b)
```

📐 Mixing Generic and Non-Generic Parameters

> A template function may accept **regular (non-template) parameters** alongside generic ones.

Example – tabulated output

```
#include <iostream>
using namespace std;

const int TABWIDTH = 8;

template <class X>
void tabOut(X data, int tab) {
    for (; tab; --tab)
        for (int i = 0; i < TABWIDTH; ++i) cout << ' ';
    cout << data << "\n";
}
int main() {
    tabOut("This is a test", 0);
    tabOut(100, 1);
    tabOut('X', 2);
    tabOut(10/3, 3);
    return 0;
}
```

Output

```
This is a test
        100
                X
                        3
```

- data is generic (X).
- tab is a standard int controlling formatting.

🚫 Restrictions on Generic Functions

> A generic function must implement **identical logic** for all types; only the data type may vary.

Counter-example

```
#include <iostream>
#include <cmath>
using namespace std;

void myfunc(int i) {
    cout << "value is: " << i << "\n";
}

void myfunc(double d) {
    double intpart, fracpart;
    fracpart = modf(d, &intpart);
    cout << "Fractional part: " << fracpart << "\n";
    cout << "Integer part: " << intpart << "\n";
}
int main() {
    myfunc(1);
    myfunc(12.2);
    return 0;
}
```

Output

```
value is: 1
Fractional part: 0.2
Integer part: 12
```

- The two overloads perform **different operations**, so they cannot be merged into a single template.

📊 Generic Sorting with Templates

> Sorting algorithms are **type-independent**, making them ideal for templates.

Bubble sort example

```
#include <iostream>
using namespace std;

template <class T>
void sort(T arr[], int size) {
    for (int i = 0; i < size - 1; ++i)
        for (int j = 0; j < size - i - 1; ++j)
            if (arr[j] > arr[j + 1]) {
                T temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
}
int main() {
    int nums[4] = {3, 1, 4, 2};
    double vals[3] = {2.2, 1.1, 3.3};

    sort(nums, 4);
    sort(vals, 3);

    cout << "Sorted int array: ";
    for (int v : nums) cout << v << ' ';
    cout << "\nSorted double array: ";
    for (double v : vals) cout << v << ' ';
    cout << endl;
    return 0;
}
```

Output

```
Sorted int array: 1 2 3 4
Sorted double array: 1.1 2.2 3.3
```

📦 Array Compaction via Templates

> Removing an element and shifting the remainder works for **any element type**.

Implementation

```
#include <iostream>
using namespace std;

template <class T>
void compact(T arr[], int size, int index) {
    for (int i = index; i < size - 1; ++i)
        arr[i] = arr[i + 1];
    arr[size - 1] = 0;      // sentinel value for illustration
}
int main() {
    int a[4] = {1, 2, 3, 4};
    compact(a, 4, 1);   // delete element at index 1
    for (int v : a) cout << v << ' ';
    return 0;
}
```

Output

```
1 3 4 0
```

🏷️ Generic (Template) Classes

> A **generic class** (class template) abstracts the class definition from the concrete data type.

Syntax

```
template <class T>
class ClassName {
    // members may use T
};
```

- Multiple type parameters: template <class T1, class T2>.

- Instantiation: ClassName<int> obj;.

Example – Pair class with two types

```
#include <iostream>
using namespace std;

template <class T1, class T2>
class MyClass {
    T1 x;
    T2 y;
public:
    MyClass(T1 a, T2 b) : x(a), y(b) {}
    void show() const { cout << x << " " << y << endl; }
};
int main() {
    MyClass<int, double> obj1(10, 23.5);
    MyClass<char, const char*> obj2('A', "Hello");
    obj1.show();
    obj2.show();
    return 0;
}
```

Output

```
10 23.5
A Hello
```

🔢 Multiple Generic Types, Non-Type Arguments & Default Parameters

Feature	Description	Example
Multiple type parameters	template<class T1, class T2> lets a class use two independent types.	See *MyClass* above.
Non-type arguments	Constant values (e.g., int size) become part of the class definition.	template<class A, int size> class Array { A data[size]; };

Default template arguments	Provide fall-backs when the user omits arguments.	template<class X=int, int size=10> class SafeArray { … };

Default-argument example

```
#include <iostream>
#include <cstdlib>
using namespace std;

template <class T=int, int size=5>
class SafeArray {
    T arr[size];
public:
    SafeArray() { for (int i=0;i<size;++i) arr[i]=i; }
    T& operator[](int i) {
        if (i<0 || i>=size) { cout<<"Index out-of-bounds.\n"; exit(1); }
        return arr[i];
    }
};

int main() {
    SafeArray<int,4> a;   // int, size 4
    SafeArray<double> b;  // double, size 5 (default)
    SafeArray<> c;        // int, size 5 (both defaults)

    for (int i=0;i<4;i++) cout<<a[i]<<' '; cout<<"\n";
    for (int i=0;i<5;i++) cout<<b[i]<<' '; cout<<"\n";
    for (int i=0;i<5;i++) cout<<c[i]<<' '; cout<<"\n";
    return 0;
}
```

Output

```
0 1 2 3
0 1 2 3 4
0 1 2 3 4
```

🧩 Explicit Class Specialization

> When a particular type needs a **different implementation**, you can **specialize** the whole class.

Syntax

```
template<> class ClassName<SpecificType> {
    // specialized members
};
```

Example

```
#include <iostream>
using namespace std;

// Generic version
template <class T>
class MyClass {
    T x;
public:
    MyClass(T a) : x(a) { cout<<"Inside generic MyClass\n"; }
    T getx() const { return x; }
};

// Specialization for int
template <>
class MyClass<int> {
    int x;
public:
    MyClass(int a) : x(a*a) { cout<<"Inside MyClass<int> specialization\n"; }
    int getx() const { return x; }
};

int main() {
    MyClass<double> d(10.1);
    cout<<"double: "<<d.getx()<<"\n\n";

    MyClass<int> i(5);
    cout<<"int: "<<i.getx()<<"\n";
    return 0;
}
```

Output

```
Inside generic MyClass
double: 10.1

Inside MyClass<int> specialization
int: 25
```

🏷️ typename and export Keywords

- **typename**
  1. Can replace class in a template declaration: template <typename X>.
  2. Disambiguates dependent names: typename X::value_type var; tells the compiler that X::value_type is a type.
- **export** *(rarely supported)*

  Allows the **definition** of a template to be placed in a separate translation unit,

  exposing only the **declaration** to other files.

```
export template <class T> class myclass;   // declaration only
```

📚 STL list Container

> list is a **doubly linked list** template that stores elements of any type.

Property	Detail
Memory	Non-contiguous; each node allocated separately
Insertion/Deletion	O(1) at any position
Random Access	Not supported (no operator[])
Typical Use	Frequent insert/erase in the middle of a sequence

Example

```
#include <iostream>
#include <list>
using namespace std;
```

```
int main() {
    list<int> mylist;
    mylist.push_back(10);
    mylist.push_back(20);
    mylist.push_front(5);

    cout << "List elements: ";
    for (int v : mylist) cout << v << ' ';
    cout << "\n";

    mylist.pop_back();
    cout << "After pop_back: ";
    for (int v : mylist) cout << v << ' ';
    cout << "\n";
    return 0;
}
```

Output

```
List elements: 5 10 20
After pop_back: 5 10
```

📈 Template Function to Find Maximum in an Array

```
#include <iostream>
using namespace std;

template <class T>
T findMax(T arr[], int size) {
    T max = arr[0];
    for (int i = 1; i < size; ++i)
        if (arr[i] > max) max = arr[i];
    return max;
}
int main() {
    int a[] = {2,4,9,1,5};
    double b[] = {1.1,5.5,3.3,9.9};

    cout << "Max in int array: " << findMax(a,5) << endl;
    cout << "Max in double array: " << findMax(b,4) << endl;
    return 0;
}
```

Output

```
Max in int array: 9
Max in double array: 9.9
```

🔄 Iterators in the Standard Template Library (STL)

> An **iterator** behaves like a pointer that traverses the elements of a container.

- **Types**: input, output, forward, bidirectional, random-access.
- **Common operations**: *it (dereference), ++it/it++ (advance), it == container.end() (termination).

Example with vector

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {1,2,3,4};
    for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
        cout << *it << " ";
    cout << endl;
    return 0;
}
```

Output

```
1 2 3 4
```

📦 vector Class Overview

Feature	Description
Dynamic resizing	Grows automatically when push_back is used.
Random access	operator[] and at() provide O(1) element retrieval.

Memory management	Handles allocation/deallocation internally.
Key member functions	push_back, pop_back, size, clear, begin, end.

Simple usage

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);

    cout << "Vector elements: ";
    for (size_t i = 0; i < v.size(); ++i) cout << v[i] << " ";
    cout << endl;
    return 0;
}
```

Output

```
Vector elements: 10 20 30
```

🏆 Template Class to Find the Largest of Three Objects

```
#include <iostream>
using namespace std;

template <class T>
class Largest {
    T x, y, z;
public:
    Largest(T a, T b, T c) : x(a), y(b), z(c) {}
    T max() const {
        T m = x;
        if (y > m) m = y;
        if (z > m) m = z;
        return m;
    }
```

```
};

int main() {
    Largest<int> li(10,50,30);
    cout << "Largest (int): " << li.max() << endl;

    Largest<double> ld(3.5,7.8,1.2);
    cout << "Largest (double): " << ld.max() << endl;
    return 0;
}
```

Output

```
Largest (int): 50
Largest (double): 7.8
```

📚 Definitions (from the PDF)

> **List** – A *sequence container* in the STL implemented as a doubly linked list. It provides fast insertion and deletion anywhere in the sequence, non-contiguous storage, and bidirectional traversal via iterators.

> **Map** – An *associative container* storing unique key-value pairs, typically implemented with a balanced binary search tree (e.g., red-black tree). Provides logarithmic search, insertion, and removal.